

SESIÓN COMPLETA SEMANA 1 — PROGRAMACIÓN SEGURA (DD281)

INTRODUCCIÓN A LA PROGRAMACIÓN SEGURA Y MODELADO DE AMENAZAS

ENCABEZADO DE LA SESIÓN

Campo	Detalle
Asignatura	Programación Segura — DD281
Ciclo	VIII — Semestre 202601
Semana	1 de 8
Modalidad	Semipresencial
Duración efectiva	150 minutos de instrucción + 2 recesos de 15 min = 180 min totales
Tema	Introducción a la Programación Segura y Modelado de Seguridad
Valor transversal	Excelencia en el diseño preventivo de vulnerabilidades de software
Docente	—
Fecha	Semana 1 — 2026-01

ALINEAMIENTO CON EL SÍLABO

Elemento	Contenido
Unidad	Unidad 1: Control de autenticación, Control de Roles y Privilegios
Temáticas del sílabo	Introducción a la Programación Segura y Modelado de Seguridad
Actividad procedimental	Trabajo Colaborativo: Modelado de Amenazas
Competencia	Aplica conceptos de programación segura en la construcción de sistemas de información
Contribución a evaluación	Formativa (diagnóstico) — Base para la EP de Semana 4

LOGRO DE APRENDIZAJE DE LA SESIÓN

"Al finalizar la sesión, el estudiante elabora un plan de desarrollo y diseña los casos de uso relacionados a la seguridad del sistema, identificando principios fundamentales de programación segura, analizando vectores de ataque en sistemas web y demostrando excelencia en el diseño preventivo de vulnerabilidades."

PARTE 1 — INICIO



Duración: 25 minutos

1.1 DINÁMICA DE APERTURA: "¿CUÁNTOS DE USTEDES YA FUERON HACKEADOS?"



8 minutos | Modalidad: Individual + plenario

Revisar este link

<https://haveibeenpwned.com>

Apertura:

"Antes de empezar con los temas del curso, quiero que hagan algo. Saquen su celular o laptop. Ingresen a la URL que está en pantalla: **haveibeenpwned.com** — y escriban el correo electrónico que más usan.

"Bien. ¿cuántos de ustedes aparecieron en alguna filtración de datos?"

ustedes no hicieron nada malo. Alguien más escribió código inseguro. Alguien más no validó entradas. Alguien más guardó contraseñas en texto plano. Y millones de personas pagaron las consecuencias. Eso es exactamente lo que este curso existe para que ustedes nunca sean 'ese alguien más'. Bienvenidos a Programación Segura."

1.2 PRESENTACIÓN DEL LOGRO DE APRENDIZAJE



3 minutos | Modalidad: Docente presenta + estudiantes reformulan

Escribe en la pizarra el logro de aprendizaje. Luego di:

1.3 ACTIVIDAD DIAGNÓSTICA: "EL MURO DE LO QUE SÉ Y LO QUE NO SÉ"

 **10 minutos** | Modalidad: Individual (2 min) → Plenario (8 min)

Instrucción: Entrega o proyecta esta tabla. Pide que la completen individualmente en 2 minutos:

DIAGNÓSTICO DE CONOCIMIENTOS PREVIOS	
¿Qué sé sobre seguridad informática?	¿Qué NO sé y me gustaría aprender en este curso?

PREGUNTA DIAGNÓSTICA 1:

"¿Alguien puede decirme qué es una vulnerabilidad de software? Dénme una definición con sus propias palabras."

RESPUESTA ESPERADA (para que el docente evalúe): Una vulnerabilidad es una debilidad, falla o defecto en el diseño, implementación o configuración de un sistema de software que puede ser explotada por un atacante para comprometer la seguridad del sistema. Por ejemplo: no validar la longitud de una contraseña, guardar datos sensibles sin cifrar, o aceptar cualquier tipo de archivo sin verificar su contenido. La respuesta del estudiante es correcta si menciona: debilidad/falla + posibilidad de ser explotada + impacto. Si solo dice "un error en el código", pide que amplíe: ¿qué tipo de error? ¿qué consecuencia tiene?

PREGUNTA DIAGNÓSTICA 2:

"¿Cuál es la diferencia entre vulnerabilidad y amenaza? ¿Son lo mismo?"

RESPUESTA ESPERADA: No son lo mismo. Una **vulnerabilidad** es la debilidad que existe en el sistema (ej: una puerta sin cerrojo). Una **amenaza** es el evento o agente que puede explotar esa debilidad (ej: un ladrón que podría entrar por esa puerta). El **riesgo** es la probabilidad de que la amenaza explote la vulnerabilidad multiplicada por el impacto que generaría (ej: qué tan probable es que ese ladrón entre, y qué tan grave sería si lo hace). La respuesta es correcta si el estudiante distingue los tres conceptos y usa alguna analogía coherente. Error común: confundir amenaza con ataque — una amenaza es potencial, un ataque es cuando ya ocurrió.

PREGUNTA DIAGNÓSTICA 3:

"¿Han escuchado hablar de OWASP? ¿Alguien sabe qué es o para qué sirve?"

RESPUESTA ESPERADA: OWASP (Open Worldwide Application Security Project) es una fundación sin fines de lucro dedicada a mejorar la seguridad del software. Su recurso más conocido es el **OWASP Top 10**, que es una lista de las 10 vulnerabilidades más críticas y frecuentes en aplicaciones web. Esta lista se actualiza periódicamente (la versión actual es 2021) y es usada globalmente como estándar de referencia por desarrolladores, auditores y equipos de seguridad. Si el estudiante dice "no sé", eso es valioso — significa que este es exactamente el punto de partida de la clase.

1.4 CONEXIÓN PROFESIONAL: "¿PARA QUÉ ME SIRVE ESTO?"

 **4 minutos | Modalidad: Lluvia de ideas rápida**

"Antes de entrar al contenido, quiero escucharles. Sin filtros: ¿para qué creen que les sirve esto en su vida profesional? ¿Cuándo lo van a usar? Díganme lo primero que les venga a la mente."

(Escribe en la pizarra lo que digan. No corrigas nada todavía.)

Respuestas que probablemente aparecerán:

- "Para no hacer aplicaciones inseguras"
- "Para trabajar en ciberseguridad"
- "Para proteger datos de usuarios"
- "Para que no me hackeen mis sistemas"

Complemento docente (después de escucharles):

"Todo eso es válido. Pero hay algo que quizás no pensaron: el 80% de los trabajos de desarrollo de software en el mercado laboral HOY exigen que el desarrollador conozca prácticas de seguridad. No como especialista, sino como parte del rol. DevSecOps es la tendencia. Ya no hay 'el equipo de seguridad por allá' y 'los desarrolladores por acá' — ahora la seguridad es responsabilidad de quien escribe el código. De ustedes."

PARTE 2 — UTILIDAD



Duración: 10 minutos

2.1 EL PROBLEMA REAL QUE RESUELVE ESTE TEMA

GUION VERBAL:

"Permítanme contarles tres hechos del mundo real. Los tres ocurrieron en los últimos años. Los tres costaron cientos de millones de dólares. Y los tres se pudieron evitar con lo que van a aprender en este semestre."

CASO 1 — Equifax (2017):

"Equifax es una de las tres agencias de crédito más grandes del mundo. Manejan datos financieros de 800 millones de personas. En 2017, sufrieron una brecha de seguridad que expuso datos de 147 millones de personas: nombres, números de seguro social, fechas de nacimiento, direcciones. ¿La causa? Una vulnerabilidad conocida en Apache Struts (CVE-2017-5638) que llevaba meses reportada y que nadie parcheó. El costo: 700 millones de dólares en multas y compensaciones. La causa técnica: no actualizar una dependencia. Algo que ustedes aprenderán a gestionar en este curso."

CASO 2 — Colonial Pipeline (2021):

"Colonial Pipeline es la compañía que abastece de combustible al 45% de la costa este de Estados Unidos. En 2021, fue víctima de un ransomware que paralizó sus operaciones durante 6 días, causando escasez de gasolina en varios estados. El vector de entrada: una contraseña de una VPN que no tenía autenticación de dos factores activada. El rescate pagado: 4.4 millones de dólares en Bitcoin. Punto de entrada: una contraseña débil sin MFA. Algo que aprenderán a implementar en la Semana 2."

CASO 3 — La base de datos universitaria expuesta:

"Un dato más cercano: en 2022, una universidad peruana fue reportada por tener su sistema de notas accesible con una inyección SQL básica. Cualquier estudiante podía acceder a las notas de sus compañeros con solo modificar un parámetro en la URL. El sistema había sido desarrollado por ex-alumnos del mismo programa. No fue malicia — fue ignorancia. Ese es exactamente el tipo de error que ustedes van a aprender a no cometer."

"¿Qué tienen en común estos tres casos? Que no fueron ataques de ciencia ficción. No requirieron hackers con superpoderes. Fueron errores de programación y diseño que cualquier desarrollador educado en seguridad podría haber evitado. Ese es el valor de este curso: no convertirlos en hackers, sino en desarrolladores que piensan como hackers para construir mejor."

PARTE 3 — TRANSFORMACIÓN



Duración: 65 minutos (Minutos 35–100)

3.1 ¿QUÉ ES LA PROGRAMACIÓN SEGURA?



10 minutos

GUION VERBAL:

"Vamos a empezar con la definición base. Y quiero que noten que no es una definición técnica aburrida — es una forma de pensar."

DEFINICIÓN FORMAL:

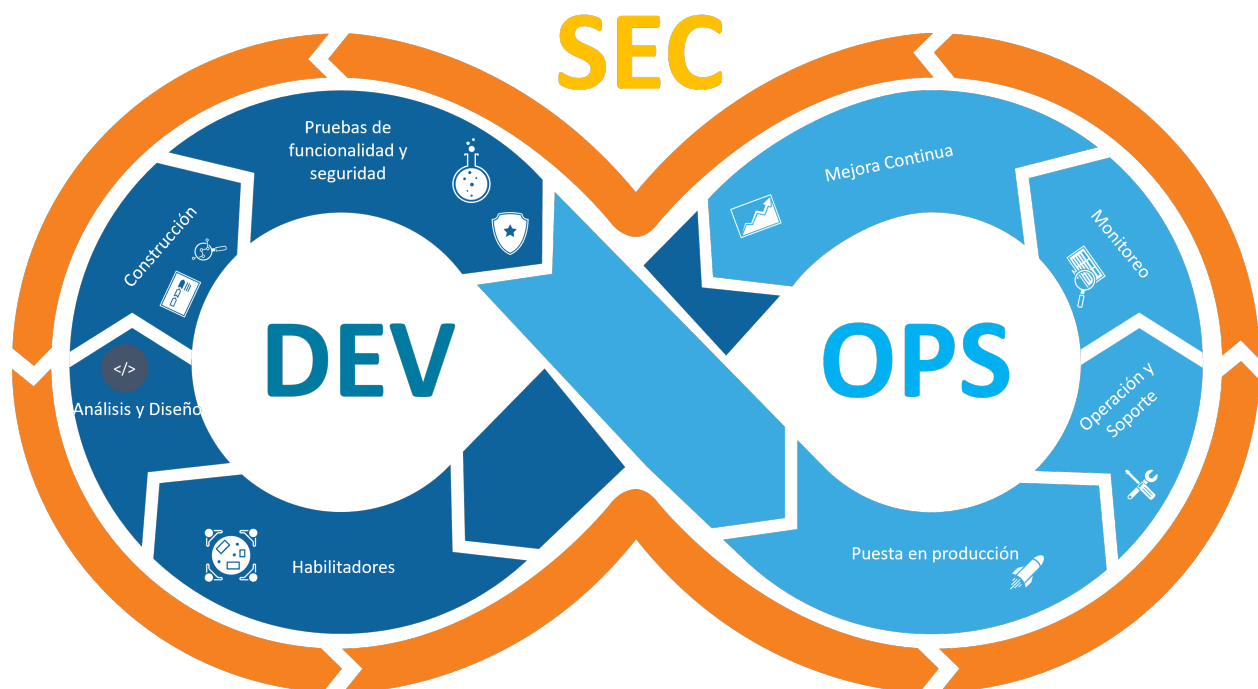
Programación Segura es el conjunto de prácticas, principios y técnicas que se aplican durante todo el ciclo de vida del desarrollo de software (SDLC) con el objetivo de minimizar vulnerabilidades, reducir la superficie de ataque y garantizar que el software se comporte de manera predecible y segura incluso ante condiciones adversas o intentos de explotación maliciosa.

PREGUNTA AL AULA 1:

"Alguien me acaba de escuchar decir 'SDLC'. Sin googlear, ¿alguien sabe qué significa y cuáles son sus fases?"

RESPUESTA ESPERADA: SDLC son las siglas de **Software Development Life Cycle** (Ciclo de Vida del Desarrollo de Software). Sus fases clásicas son: (1) Planificación/Requerimientos, (2) Análisis, (3) Diseño, (4) Implementación/Codificación, (5) Pruebas/Testing, (6) Despliegue, (7) Mantenimiento. La respuesta correcta debe incluir al menos las fases de análisis, diseño, implementación y pruebas. Si el estudiante menciona solo "análisis, diseño, codificación y pruebas", es aceptable. El punto clave a reforzar: **en la programación segura, la seguridad se integra en TODAS las fases, no solo en las pruebas**. Error común: creer que la seguridad es solo una fase al final (testing de penetración). Eso es tardío y costoso.

Ciclo de Vida del Desarrollo de Software (SDLC)



Fases del SDLC

Aunque no definiremos cada fase en detalle, aquí tienes una visión general de cómo estructuramos nuestro SDLC:

Fase	Descripción
Plan	Definición de objetivos y alcance del proyecto.
Code	Codificación y construcción del software.
Build	Compilación y construcción de artefactos de software.
Test	Verificación y validación del software.
Release	Preparación del software para su lanzamiento.
Deploy	Implementación del software en el entorno de producción.
Operate	Operación y gestión del software en producción.
Monitor	Monitoreo continuo del rendimiento y funcionamiento del software.

Complemento docente — La regla del "costo de corregir":

Dibuja esta pirámide invertida en la pizarra:

COSTO DE CORREGIR UNA VULNERABILIDAD

Requerimientos	→ \$1
Diseño	→ \$10
Codificación	→ \$100
Pruebas	→ \$1,000
Producción	→ \$10,000 o más

"Esta es la regla más importante de la programación segura y la más ignorada en la industria. Cuanto más tarde encuentras un error de seguridad, más caro es corregirlo. IBM lo documentó: un bug encontrado en producción cuesta entre 6 y 100 veces más que uno encontrado en la fase de diseño. Por eso este curso existe: para que piensen en seguridad ANTES de escribir la primera línea de código."

3.2 LA TRIADA CIA: EL CORAZÓN DE LA SEGURIDAD

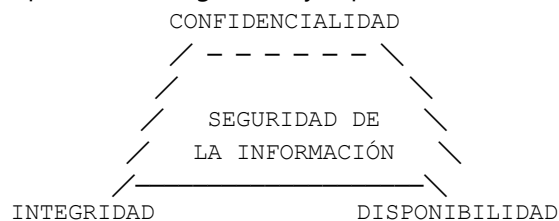
🕒 12 minutos

PREGUNTA AL AULA 2:

"¿Alguien conoce la Triada CIA de la seguridad informática? No me refiero a la agencia de inteligencia. ¿Qué son esas tres letras en seguridad?"

RESPUESTA ESPERADA: CIA son las siglas de **Confidentiality** (Confidencialidad), **Integrity** (Integridad) y **Availability** (Disponibilidad). Son los tres pilares fundamentales de la seguridad de la información. Un estudiante que mencione los tres conceptos básicos tiene la respuesta correcta. Si no saben, es el punto de partida perfecto para enseñar.

Dibuja en la pizarra el triángulo CIA y explica cada vértice con un ejemplo concreto:



CONFIDENCIALIDAD:

"La información solo debe ser accesible por quienes están autorizados para verla. Ejemplo: el historial médico de un paciente solo puede verlo su médico tratante, no el recepcionista de la clínica, no el personal de limpieza. En código: significa cifrar datos sensibles, controlar accesos, no exponer información en logs."

PREGUNTA AL AULA 3:

"¿Qué pasaría si un sistema de e-commerce no garantiza la confidencialidad de los datos de tarjetas de crédito? Denme un escenario concreto de consecuencias."

RESPUESTA ESPERADA: Si un e-commerce no protege la confidencialidad de los datos de tarjetas de crédito: (1) Un atacante podría acceder a la base de datos y robar los números de tarjeta, fechas de vencimiento y CVV. (2) Esos datos se venden en mercados negros (dark web) por entre \$5 y \$50 por tarjeta. (3) Los titulares sufren cargos no autorizados. (4) La empresa enfrenta multas PCI-DSS que pueden llegar a \$100,000 por mes. (5) La reputación de la empresa se destruye — el 60% de las pequeñas empresas cierran dentro de 6 meses de sufrir una brecha de datos. La respuesta debe incluir al menos el daño al usuario final Y el daño a la empresa. Si solo mencionan uno, pide que amplíen.

INTEGRIDAD:

"Los datos deben mantenerse exactos y completos, y solo pueden ser modificados por procesos o personas autorizadas. Ejemplo: en un sistema bancario, nadie debe poder modificar el saldo de su cuenta a voluntad. En código: significa usar hashes para verificar que los archivos no han sido alterados, usar firmas digitales, validar que los datos no han sido manipulados en tránsito."

PREGUNTA AL AULA 4:

"¿Qué ataque conocen que viola la integridad de los datos? Piensen en ataques a sistemas web."

RESPUESTA ESPERADA: Los principales ataques que violan la integridad son: (1) **SQL Injection:** el atacante modifica registros de la base de datos. Por ejemplo, cambia el precio de un producto de \$100 a \$0.01. (2) **Inyección de comandos:** ejecuta comandos del sistema operativo que modifican archivos. (3) **Man-in-the-Middle (MITM):** intercepta los datos en tránsito y los modifica antes de que lleguen al destinatario. (4) **Defacement de sitios web:** el atacante modifica el contenido visible de una web. (5) **Manipulación de parámetros:** modificar el valor de un parámetro en la URL para alterar el comportamiento del sistema (ej: `precio=100` → `precio=0`). Es correcto si mencionan al menos dos ataques con explicación de cómo violan la integridad.

DISPONIBILIDAD:

"El sistema y sus datos deben estar accesibles para los usuarios autorizados cuando los necesiten. Ejemplo: un hospital que no puede acceder a los registros de un paciente en emergencias por culpa de un ransomware puede costar vidas humanas. En código: significa diseñar sistemas resilientes, con manejo de errores, sin puntos únicos de falla, con protección contra DDoS."

PREGUNTA AL AULA 5:

"El ataque más común contra la disponibilidad es el DDoS. ¿Alguien puede explicar cómo funciona un ataque de Denegación de Servicio Distribuido?"

RESPUESTA ESPERADA: Un ataque DDoS (Distributed Denial of Service) funciona así: (1) El atacante toma control de miles o millones de dispositivos infectados (computadoras, routers, cámaras IoT) — estos forman una **botnet**. (2) Da la orden a todos estos dispositivos de enviar solicitudes simultáneas al servidor víctima. (3) El servidor, al recibir millones de solicitudes por segundo, supera su capacidad de procesamiento. (4) El servidor se satura y deja de responder a solicitudes legítimas. (5) El servicio queda "caído" para los usuarios reales. Los ataques DDoS más grandes registrados han superado los 3 Terabits por segundo (Google 2017, Cloudflare 2022). La respuesta correcta debe explicar el concepto de botnet + saturación del servidor + impacto en usuarios legítimos.

3.3 VOCABULARIO FUNDAMENTAL DE SEGURIDAD

10 minutos

"Antes de seguir, necesitamos hablar el mismo idioma. Estos 5 conceptos van a aparecer en cada semana del curso. Si los tienen claros, todo lo demás encaja."

Escribe en la pizarra esta estructura y ve construyéndola con participación estudiantil:

CONCEPTO	DEFINICIÓN	EJEMPLO
ACTIVO	Recurso de valor para la organización	BD de clientes, código fuente
VULNERAB.	Debilidad explotable	SQL sin sanitizar
AMENAZA	Evento potencial dañino	Hacker, insider
RIESGO	Probabilidad × Impacto	Nivel: Alto/Medio
CONTROL	Medida que reduce riesgo	Autenticación MFA

PREGUNTA AL AULA 6 (para construir la tabla juntos):

"Piensen en un sistema universitario de matrícula online. ¿Cuáles serían los activos más críticos de ese sistema? Quiero escuchar al menos cuatro."

RESPUESTA ESPERADA: Los activos críticos de un sistema de matrícula universitaria son: (1) **Base de datos de estudiantes:** datos personales, historial académico, datos de contacto. (2) **Registros de notas:** información confidencial de rendimiento académico. (3) **Datos de pago:** información de transacciones de matrícula, métodos de pago registrados. (4) **Credenciales de acceso:** usuarios y contraseñas del sistema. (5) **Horarios y asignación de aulas:** información operativa crítica. (6) **Identidades de docentes y administrativos:** sus cuentas tienen privilegios elevados. (7) **El código fuente del sistema:** si un atacante lo obtiene, puede encontrar vulnerabilidades específicas. La respuesta es buena si el estudiante categoriza entre activos de datos, activos de sistema y activos intangibles.

3.4 OWASP TOP 10 — EL MAPA DE LOS ENEMIGOS

 15 minutos

"OWASP publica cada pocos años la lista de las 10 vulnerabilidades más críticas en aplicaciones web. Esta lista es tan importante que muchos contratos de desarrollo de software exigen que el sistema la cumpla. La versión más reciente es la de 2021."

Proyecta o escribe en la pizarra:

OWASP TOP 10 — 2021

A01. Broken Access Control	(Control de acceso roto)
A02. Cryptographic Failures	(Fallos criptográficos)
A03. Injection	(Inyección: SQL, NoSQL, OS)
A04. Insecure Design	(Diseño inseguro)
A05. Security Misconfiguration	(Mala configuración)
A06. Vulnerable Components	(Componentes vulnerables)
A07. Identification/Auth Failures	(Fallos de autenticación)
A08. Software/Data Integrity Fails	(Integridad de datos)
A09. Security Logging Failures	(Fallos de registro/monitoreo)
A10. Server-Side Request Forgery	(SSRF)

PREGUNTA AL AULA 7:

"Miren la lista. Tenemos solo 2 minutos. Intuitivamente, sin que yo les haya explicado nada, ¿cuál creen que es la #1 más explotada hoy en día y por qué?"

RESPUESTA ESPERADA: La respuesta correcta del OWASP es que **A01: Broken Access Control** subió al número 1 en 2021 (antes estaba en el puesto 5). El 94% de las aplicaciones probadas presentaban alguna forma de control de acceso roto. Esto significa que los sistemas permiten a usuarios hacer cosas que no deberían: acceder a datos de otros usuarios, ejecutar funciones administrativas sin ser admin, ver URLs que deberían estar restringidas. Si un estudiante dice "Inyección (A03)" porque es la más conocida históricamente, es una respuesta comprensible pero desactualizada — en 2021, SQL Injection bajó de puesto 1 a puesto 3. Aprovecha esto para enseñar que la amenaza más grave cambia con el tiempo.

MINI-CASO de A03 — Injection (para demostrar en vivo):

"Voy a mostrarles por qué A03 — Injection sigue siendo mortal. Este es el código más peligroso que pueden escribir en Java:"

```
// == CÓDIGO INSEGURO — SQL INJECTION ==  
// NUNCA hagan esto en producción  
String usuario = request.getParameter("username");  
String sql = "SELECT * FROM usuarios WHERE nombre = '" + usuario + "'";  
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery(sql);  
// Si el atacante escribe: ' OR '1'='1  
// El SQL resultante es: SELECT * FROM usuarios WHERE nombre = '' OR '1'='1'  
// Resultado: accede a TODOS los registros de la tabla
```

"¿Ven el problema? El atacante ingresa ' OR '1'='1 en el campo de usuario y el sistema le devuelve todos los registros. No necesita saber ninguna contraseña. Solo necesita saber que el developer no validó la entrada. La solución — que aprenderán en profundidad en la Semana 5 — se llama PreparedStatement:"

```
// == CÓDIGO SEGURO — PreparedStatement ==  
String sql = "SELECT * FROM usuarios WHERE nombre = ?";  
PreparedStatement ps = conn.prepareStatement(sql);  
ps.setString(1, usuario); // El parámetro se trata como DATO, no como código  
ResultSet rs = ps.executeQuery();  
// El atacante puede escribir lo que quiera — será tratado como texto, no como SQL
```

PREGUNTA AL AULA 8:

"¿Por qué el PreparedStatement evita la inyección SQL? ¿Qué hace diferente internamente?"

RESPUESTA ESPERADA: El PreparedStatement previene SQL Injection porque separa el **código SQL** de los **datos del usuario** en dos etapas distintas. En el PreparedStatement: (1) Primero se compila el SQL con el ? como marcador de posición — en esta etapa el motor de base de datos ya interpreta la estructura del query. (2) Luego se asignan los valores con `setString()`, `setInt()`, etc. — estos valores siempre se tratan como datos literales, nunca como código SQL ejecutable. Entonces si el atacante escribe ' OR '1'='1, ese string completo se busca literalmente en la columna "nombre" — ningún usuario se llama así, así que no devuelve resultados. La diferencia clave: con concatenación, el motor SQL interpreta el input como parte de la instrucción; con PreparedStatement, el motor SQL ya tiene la instrucción compilada y solo recibe un valor de dato.

3.5 LOS 8 PRINCIPIOS DE PROGRAMACIÓN SEGURA

 **10 minutos**

"Existen principios que guían las decisiones de seguridad. No son reglas arbitrarias — son lecciones aprendidas de miles de brechas de seguridad. Los van a usar en cada semana del curso."

Construye esta lista con participación — pregunta antes de explicar:

PREGUNTA AL AULA 9:

"El primer principio se llama 'Mínimo Privilegio'. Solo con el nombre, ¿qué creen que significa y por qué existe?"

RESPUESTA ESPERADA: El principio de **Mínimo Privilegio** (Least Privilege) establece que cada usuario, proceso o componente del sistema debe tener únicamente los permisos estrictamente necesarios para realizar su función, y nada más. Existe porque si una cuenta o componente tiene más permisos de los necesarios y es comprometida, el daño que puede causar el atacante es proporcional a esos permisos. Ejemplo: si el usuario de base de datos de una aplicación web solo necesita hacer SELECT e INSERT, no debe tener permisos de DROP o DELETE. Si ese usuario es comprometido, el atacante no podrá borrar tablas. La respuesta es correcta si incluye: solo los permisos necesarios + razón de existir (limitar el daño potencial). Error común: confundir "mínimo privilegio" con "sin privilegios" — no se trata de restringir todo, sino de otorgar exactamente lo necesario.

Presenta los 8 principios en tabla:

#	PRINCIPIO	EJEMPLO PRÁCTICO
1	Mínimo Privilegio	BD: usuario app solo con SELECT
2	Defensa en Profundidad	Login + MFA + Rate limit + Log
3	Fallo Seguro	Si falla auth → acceso denegado
4	No confiar en el cliente	Validar en servidor, no solo JS
5	Separación de privilegios	Admin ≠ Usuario ≠ Sistema
6	Economía del mecanismo	Simple es más seguro que complejo
7	Mediación completa	Verificar permisos en cada acceso
8	Seguridad por diseño	Modelar amenazas antes de codear

PREGUNTA AL AULA 10 — La más desafiante de la sección:

"El principio #3 dice 'Fallo Seguro'. ¿Qué significa que un sistema 'falle de forma segura'? ¿Cuál sería un ejemplo de un sistema que falla de forma INsegura?"

RESPUESTA ESPERADA: Fallo seguro (Fail Secure o Fail Safe) significa que cuando un sistema encuentra un error, una excepción o una situación no prevista, su respuesta por defecto debe ser **denegar el acceso** o **mantener el sistema en un estado seguro**, nunca abrir puertas por error. Ejemplo de fallo seguro: si el servicio de autenticación está caído, el sistema dice "No se puede acceder temporalmente" — no permite el acceso sin autenticación. Ejemplo de fallo INseguro: un sistema de torniquetes de metro que, cuando pierde conexión con el servidor de validación, abre todos los torniquetes por defecto para no interrumpir el servicio — esto permite que personas sin pagar entren libremente. Otro ejemplo de fallo inseguro en código: un bloque `catch` vacío que captura una excepción de autenticación y por no manejarla correctamente permite el acceso: `catch(Exception e) { /* silencio */ loginExitoso = true; }`. La respuesta correcta debe incluir: qué hace en caso de error + por qué denegar es lo seguro.

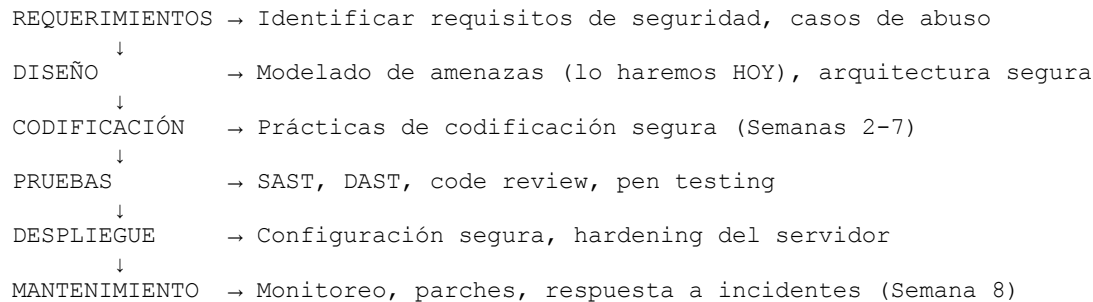
3.6 EL SDLC SEGURO — INTEGRAR SEGURIDAD EN TODO EL PROCESO

 8 minutos

"Hemos hablado de qué es la seguridad. Ahora hablaremos de CUÁNDO aplicarla. La respuesta corta es: siempre. La respuesta larga es el SDLC seguro."

Dibuja este diagrama en la pizarra:

SDLC SEGURO — Actividades de seguridad por fase



PREGUNTA AL AULA 11:

"Si tuvieran que elegir SOLO UNA fase del SDLC donde invertir más tiempo en seguridad, ¿cuál elegirían y por qué? Tengan en cuenta el costo de corregir que vimos antes."

RESPUESTA ESPERADA: La respuesta técnicamente más fundamentada es la fase de **DISEÑO** (específicamente el modelado de amenazas). La razón: en la fase de diseño todavía no se ha escrito código, por lo que cambiar la arquitectura para agregar un control de seguridad cuesta tiempo de papel y lápiz, no horas de refactorización. Si se detecta en diseño que el sistema necesita autenticación en dos pasos, se puede añadir al diagrama en minutos. Si se detecta en producción con miles de usuarios activos, requiere rediseñar el flujo de autenticación, modificar el código, hacer pruebas de regresión, coordinar el despliegue y potencialmente migrar sesiones activas. Esto puede tomar semanas y cuesta 100x más. Algunos estudiantes pueden argumentar "requerimientos" — también es válido porque si no se definen los requisitos de seguridad desde el inicio, el problema viene de raíz. Acepta ambas respuestas si la argumentación es coherente.

3.7 MODELADO DE AMENAZAS — LA METODOLOGÍA STRIDE

10 minutos

"Llegamos al concepto más importante de la sesión de hoy. El Modelado de Amenazas es la práctica de pensar SISTEMÁTICAMENTE como un atacante para identificar posibles puntos débiles ANTES de construir el sistema. La metodología más usada se llama STRIDE y fue creada por Microsoft."

S — SPOOFING → Suplantación de identidad
T — TAMPERING → Manipulación de datos
R — REPUDIATION → Negación de acciones
I — INFORMATION DISC. → Divulgación de información
D — DENIAL OF SERVICE → Denegación de servicio
E — ELEVATION OF PRIV → Escalada de privilegios

PREGUNTA AL AULA 12:

"Tenemos un sistema de login universitario básico: formulario de usuario y contraseña que consulta una base de datos. Usando STRIDE, ¿cuál de las 6 categorías representa la amenaza más inmediata y por qué?"

RESPUESTA ESPERADA: Para un sistema de login universitario básico, las amenazas más inmediatas son: (1) **Spoofing (S)**: Un atacante puede suplantar la identidad de otro estudiante si las credenciales son débiles o fueron filtradas. Esto es inmediato porque muchos usuarios usan contraseñas sencillas como su número de matrícula. (2) **Information Disclosure (I)**: Si el sistema muestra mensajes de error diferentes para "usuario no existe" vs "contraseña incorrecta", permite enumerar qué usuarios existen en el sistema. (3) **Denial of Service (D)**: Ataques de fuerza bruta sin límite de intentos pueden bloquear el acceso legítimo de estudiantes durante períodos de matrícula. No hay una única respuesta "correcta" — evalúa si el razonamiento es sólido. La respuesta excelente menciona más de una categoría STRIDE y justifica por qué para ESE sistema específico son las más relevantes.

RECESO 1 — 15 MINUTOS

(Minutos 100–115)

"Tomamos 15 minutos. Antes de que se levanten: en el receso, piensen en un sistema que conozcan — puede ser el sistema de su universidad, una app que usen, cualquiera — e identifiquen una posible amenaza usando STRIDE. Cuando volvamos, lo compartirán."

PARTE 4 — PRÁCTICA



Duración: 40 minutos (Minutos 115–155)

4.1 CASO PRÁCTICO: MODELADO DE AMENAZAS "RETAILPLUS"



25 minutos | Modalidad: Equipos de 4 estudiantes

CONTEXTO DEL CASO (el docente lo lee en voz alta):

"RetailPlus es una empresa peruana de comercio electrónico que vende electrónicos en línea. Tiene: un portal web para clientes, un módulo de carrito y pagos, un panel de administración para gestionar inventario y pedidos, una base de datos con información de 50,000 clientes (nombres, emails, teléfonos, historiales de compra), y un módulo de pagos conectado a Visa/MasterCard. El CTO de RetailPlus les ha contratado para hacer el modelado de seguridad de su sistema ANTES de lanzarlo al mercado. Tienen 20 minutos."

INSTRUCCIÓN PARA LOS EQUIPOS:

Entrega o proyecta esta plantilla. Cada equipo debe completarla para el caso RetailPlus:

PLANTILLA DE MODELADO DE AMENAZAS — RetailPlus				
TABLA 1: ACTIVOS CRÍTICOS				
N°	Activo	Tipo	Críticidad	
1				
2				
3				
TABLA 2: AMENAZAS IDENTIFICADAS (usar STRIDE)				
N°	Amenaza	Categoría	Prob.	Impacto
1				
2				
3				
TABLA 3: CONTROLES PROPUESTOS				
N°	Amenaza que mitiga	Control de seguridad		
1				
2				

PREGUNTAS DE ANDAMIAJE (el docente las hace mientras circula por los equipos):

Pregunta de andamiaje 1:

"¿Han identificado los activos de datos separados de los activos de sistema? Un activo de dato es la información; un activo de sistema es el componente que la procesa. ¿Cuál es más crítico en RetailPlus?"

Respuesta esperada para el docente: En RetailPlus, los activos de datos más críticos son: datos de tarjetas de crédito (críticidad: CRÍTICA — regulación PCI-DSS), base de datos de clientes (críticidad: ALTA — privacidad y GDPR), credenciales de administrador (críticidad: CRÍTICA — compromete todo el sistema). Los activos de sistema más críticos: servidor de pagos (críticidad: CRÍTICA), panel de administración (críticidad: ALTA), servidor de base de datos (críticidad: CRÍTICA). En términos de impacto para un e-commerce, los datos de tarjetas son los más críticos porque tienen consecuencias regulatorias directas (multas PCI-DSS) además del daño a los usuarios.

Pregunta de andamiaje 2:

"Para el módulo de pagos, ¿qué categoría de STRIDE aplica cuando alguien intenta modificar el monto de una transacción?"

Respuesta esperada para el docente: La manipulación del monto de una transacción es **Tampering (T)** — manipulación de datos. Específicamente, si el monto viaja en un parámetro de formulario o en la URL sin protección, un atacante puede interceptar la solicitud con Burp Suite y cambiar `precio=299.99` a `precio=0.01`. El control para mitigar esto es: nunca confiar en el precio que viene del cliente — siempre recalcular el precio en el servidor consultando la base de datos del catálogo. Adicionalmente, usar HTTPS

para cifrar la transmisión (mitiga el sniffing) y firmar los valores de transacción con HMAC para detectar manipulaciones.

Presentación de resultados (5 minutos): Pide a cada equipo que comparta su amenaza más crítica y el control propuesto. Compara con la solución modelo de tu hoja de soluciones (que ya tienes en el archivo SOLUCIONES_DOCENTE_S1).

4.2 DISEÑO DE UN CASO DE USO SEGURO

 **15 minutos | Modalidad: Individual o en pareja**

"Un caso de uso seguro no es solo describir qué hace el sistema — es describir qué hace el sistema CUANDO las cosas salen mal. Incluye flujos alternativos de seguridad, precondiciones de seguridad y postcondiciones."

Presenta esta plantilla de caso de uso seguro:

PLANTILLA: CASO DE USO SEGURO
Identificador : CU-01 Nombre : Autenticación de usuario en RetailPlus Actor(es) : Cliente registrado / Sistema de autenticación
PRECONDICIONES DE SEGURIDAD 1. La conexión debe ser HTTPS (SSL/TLS activo) 2. El usuario no debe estar bloqueado por intentos previos 3. El token CSRF debe estar presente en el formulario
FLUJO PRINCIPAL 1. Usuario ingresa email y contraseña 2. Sistema valida formato del email (regex) 3. Sistema consulta BD con PreparedStatement 4. Sistema compara contraseña con hash BCrypt almacenado 5. Si OK: genera token de sesión seguro, registra en log 6. Sistema redirige al dashboard del usuario
FLUJOS ALTERNATIVOS DE SEGURIDAD FA-1: Credenciales incorrectas → Sistema muestra mensaje GENÉRICO (no indica cuál falló) → Incrementa contador de intentos para esa IP → Si intentos ≥ 3: bloqueo temporal de 15 minutos FA-2: Cuenta bloqueada → Mensaje genérico + opción de recuperación por email FA-3: Sesión expirada → Redirigir al login con mensaje claro
POSTCONDICIONES DE SEGURIDAD 1. El evento de login queda registrado en log de auditoría 2. El token de sesión tiene tiempo de expiración definido 3. La contraseña NUNCA viaja ni se almacena en texto plano
AMENAZAS MITIGADAS POR ESTE CASO DE USO → SQL Injection (PreparedStatement) → Fuerza bruta (rate limiting + bloqueo) → Enumeración de usuarios (mensaje genérico) → Robo de sesión (token seguro con expiración)

TAREA EN CLASE (individual o en pareja, 10 minutos):

"Usando esta plantilla como modelo, diseñen el caso de uso seguro para 'Recuperación de contraseña' en RetailPlus. Piensen en qué puede salir mal en cada paso y qué haría el sistema para protegerse."

PREGUNTA AL AULA 13 — Para guiar la tarea:

"En el flujo de recuperación de contraseña, ¿por qué es un riesgo de seguridad que el sistema diga 'El email ingresado no está registrado'?"

RESPUESTA ESPERADA: Este es un problema de **enumeración de usuarios** (User Enumeration). Si el sistema dice "El email no está registrado", el atacante puede usar esto para confirmar qué emails sí tienen cuenta en el sistema. El proceso del ataque: (1) El atacante tiene una lista de miles de emails de una filtración previa. (2) Los prueba uno por uno en el formulario de recuperación. (3) Los que reciben "Email no registrado" — descartados. (4) Los que reciben "Te enviamos un enlace" o cualquier respuesta diferente — confirmados como usuarios del sistema. (5) Ahora el atacante tiene una lista validada de usuarios reales del sistema, que puede usar para ataques dirigidos. La solución: el sistema siempre debe responder lo mismo, independientemente de si el email existe o no: *"Si ese correo está registrado, recibirás un enlace de recuperación en los próximos minutos."*

PARTE 5 — CIERRE



Duración: 20 minutos (Minutos 160–180)

5.1 SÍNTESIS COLABORATIVA: "LA PIZARRA DE HOY"



8 minutos

"Vamos a construir juntos el resumen de la sesión. Yo borro la pizarra y ustedes me dicen qué escribo. Regla: cada concepto que mencionan debe venir con un ejemplo. Empezamos."

Ve escribiendo en la pizarra lo que los estudiantes digan. Si alguien dice solo el concepto sin ejemplo, di:

"Bien, ¿y me dan un ejemplo de eso en un sistema real?"

El docente guía hacia estas ideas clave si no aparecen solas:

- La triada CIA y qué viola cada pilar
 - La diferencia entre vulnerabilidad / amenaza / riesgo
 - Por qué la seguridad se integra desde el diseño, no al final
 - La metodología STRIDE para modelar amenazas
 - El caso de uso seguro como artefacto de diseño
-

5.2 METACOGNICIÓN

 **5 minutos**

Proyecta estas 3 preguntas y pide que las respondan en silencio, por escrito, en su cuaderno (no se recogen — son para ellos):

METACOGNICIÓN — ¿QUÉ APRENDÍ HOY?

1. ¿Cuál fue el concepto que más me costó entender hoy y por qué?
2. ¿Qué haría diferente en un proyecto que ya hice o estoy haciendo, después de lo que aprendí hoy?
3. ¿Qué pregunta me quedó sin responder y quiero investigar antes de la próxima semana?

Luego di:

"Quien quiera compartir alguna respuesta, con gusto escuchamos. No es obligatorio — pero quien lo haga, les garantizo que fija mejor el aprendizaje."

(Espera voluntarios. Comenta brevemente cada respuesta compartida.)

5.3 PREGUNTA FINAL DE REFLEXIÓN ÉTICA

 **2 minutos**

"Antes de cerrar, una pregunta que no tiene respuesta única. Quiero que la piensen esta semana:"

"Si ustedes desarrollan un sistema y más tarde descubren que tiene una vulnerabilidad de seguridad que podría exponer datos de miles de usuarios, ¿tienen la obligación ética — y legal — de reportarlo aunque eso signifique reconocer que cometieron un error? ¿A quién se lo reportan? ¿Cómo lo manejarían?"

(No pides respuesta ahora — lo retomas en la Semana 8 cuando hablen de monitoreo y respuesta a incidentes.)

5.4 TAREA PARA LA SIGUIENTE SEMANA

 **3 minutos**

"Para la Semana 2 necesitan traer:"

TAREA 1 — Individual (30 minutos máximo): Leer el documento del OWASP Top 10 2021 (páginas 1-20). Disponible gratuitamente en:

<https://owasp.org/Top10/>

Identificar: para CADA una de las 10 vulnerabilidades, una empresa real que haya sido afectada por esa categoría de vulnerabilidad. Traer anotado en su cuaderno o documento digital.

TAREA 2 — Grupal (avance del proyecto — 1 hora): Tomando la decisión del proyecto que tomaron hoy en clase, completar:

- Lista de activos críticos de su sistema
- Primera versión del diagrama de flujo de datos (DFD) del sistema
- Identificación de al menos 5 amenazas usando STRIDE

"En la Semana 2 comenzamos a implementar el módulo de autenticación segura. Necesitan tener el modelado de amenazas de su proyecto avanzado para poder decidir qué controles implementar."

5.5 SÍNTESIS FINAL DEL DOCENTE

 2 minutos

"Cierro con esto:"

"Hoy aprendieron que la seguridad no es un feature — es una propiedad del sistema que se construye desde el primer momento de diseño. Aprendieron que las vulnerabilidades no las crean los hackers — las creamos nosotros cuando escribimos código sin pensar en qué puede salir mal. Aprendieron la triada CIA, los principios de programación segura, el OWASP Top 10 y la metodología STRIDE para modelar amenazas de forma sistemática."

"Pero más importante que todos esos conceptos: aprendieron a pensar como adversario. Esa habilidad — preguntarse '¿cómo rompería yo este sistema?' — es lo que distingue a un desarrollador promedio de un desarrollador excelente. Eso es lo que los hace más valiosos en el mercado. Eso es lo que construimos juntos en estas 8 semanas."

"Nos vemos la próxima semana. Cuídense."

CASOS REALES RECOMENDADOS PARA ESTA SESIÓN

Caso	Año	Vulnerabilidad	Impacto	Link de referencia
Equifax	2017	Apache Struts sin parche (CVE-2017-5638)	147M personas afectadas, \$700M multa	https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement
Colonial Pipeline	2021	Contraseña débil + sin MFA en VPN	6 días sin combustible, \$4.4M rescate	https://www.cisa.gov/news-events/news/attack-colonial-pipeline
Yahoo! (2013-2014)	2016	Hashing débil (MD5) de contraseñas	3,000 millones de cuentas	Reportado en news.ycombinator.com

Caso	Año	Vulnerabilidad	Impacto	Link de referencia
			comprometidas	
Log4Shell	2021	Vulnerabilidad en Log4j (CVSS: 10.0)	Millones de servidores vulnerables	https://nvd.nist.gov/vuln/detail/CVE-2021-44228
SolarWinds	2020	Supply chain attack	18,000 organizaciones afectadas	https://www.cisa.gov/supply-chain-compromise

CÓDIGOS Y HERRAMIENTAS APLICABLES

Código 1 — Validación de contraseña segura en Java

(Para mostrar en clase durante la Transformación)

```
/**
 * Valida si una contraseña cumple con los criterios mínimos de seguridad.
 * Principio aplicado: Validación de entrada en el lado del servidor.
 * Referencia: NIST SP 800-63B
 */
public class ValidadorContrasena {

    public static boolean esContrasenaSegura(String contrasena) {
        // Principio: Fallo seguro – si es null, retornar false inmediatamente
        if (contrasena == null) return false;

        // Criterio 1: Longitud mínima (NIST recomienda mínimo 8 caracteres)
        if (contrasena.length() < 8) {
            System.out.println("❌ Muy corta: mínimo 8 caracteres");
            return false;
        }

        // Criterio 2: Debe contener al menos un dígito
        boolean tieneDígito = contrasena.matches(".*[0-9].*");

        // Criterio 3: Debe contener al menos una mayúscula
        boolean tieneMayúscula = contrasena.matches(".*[A-Z].*");

        // Criterio 4: Debe contener al menos un símbolo especial
        boolean tieneSimboloEspecial = contrasena.matches(".*[!@#$%^&*()_+=-].*");

        // Principio: Fallo seguro – denegar si NO cumple TODOS los criterios
        if (!tieneDígito || !tieneMayúscula || !tieneSimboloEspecial) {
            System.out.println("❌ Contraseña no cumple criterios de complejidad");
            return false;
        }

        System.out.println("✅ Contraseña segura");
        return true;
    }
}
```

```

    }

    public static void main(String[] args) {
        // Casos de prueba
        System.out.println("--- Probando contraseñas ---");
        esContrasenaSegura("password");           // ✗ Sin mayúscula ni símbolo
        esContrasenaSegura("Password1");          // ✗ Sin símbolo especial
        esContrasenaSegura("P@ss1");              // ✗ Muy corta
        esContrasenaSegura("Programacion$egura2024"); // ✔ Cumple todos los criterios
    }
}

```

¿Qué aprenden con este código?

- Validación de entrada en el servidor (nunca solo en JavaScript del cliente)
- Principio de fallo seguro (null → false)
- Criterios de complejidad de contraseña según NIST SP 800-63B
- Por qué cada verificación es necesaria

Código 2 — Comparación: SQL Injection vs PreparedStatement

```

import java.sql.*;

public class ComparacionSQL {

    // =====
    // VERSIÓN INSEGURA — NUNCA USAR EN PRODUCCIÓN
    // Vulnerabilidad: SQL Injection (OWASP A03:2021)
    // =====
    public void loginInseguro(Connection conn, String usuario, String contrasena)
        throws SQLException {

        // Si usuario = "admin'--" → SQL resultante:
        // SELECT * FROM usuarios WHERE email='admin'--' AND contrasena='...'
        // El '--' comenta el resto → accede sin conocer la contraseña
        String sql = "SELECT * FROM usuarios WHERE email='"
            + usuario + "' AND contrasena='" + contrasena + "'";

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql);

        if (rs.next()) {
            System.out.println("INSEGURO: Acceso concedido a: " +
                rs.getString("nombre"));
        }
    }

    // =====
    // VERSIÓN SEGURA — PreparedStatement + Hash de contraseña
    // Mitiga: SQL Injection, almacenamiento de contraseñas en plano
    // =====
    public void loginSeguro(Connection conn, String email, String inputContrasena)
        throws SQLException {

```

```

// El '?' es un marcador de posición – se trata siempre como DATO, nunca como
código String sql = "SELECT contraseña_hash, nombre FROM usuarios WHERE email = ?";

PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, email); // El input del usuario se pasa como parámetro

ResultSet rs = ps.executeQuery();

if (rs.next()) {
    String hashAlmacenado = rs.getString("contraseña_hash");

    // En producción: usar BCrypt.checkpw(inputContraseña, hashAlmacenado)
    // Aquí simplificado para el ejemplo didáctico
    if (verificarConBCrypt(inputContraseña, hashAlmacenado)) {
        System.out.println("SEGURO: Acceso concedido a: " +
rs.getString("nombre"));
    } else {
        // Mensaje GENÉRICO – no revela si fue el email o la contraseña
        System.out.println("SEGURO: Credenciales inválidas. Intente
nuevamente.");
    }
} else {
    // Mismo mensaje aunque el email no exista – evita enumeración de usuarios
    System.out.println("SEGURO: Credenciales inválidas. Intente nuevamente.");
}
}

private boolean verificarConBCrypt(String input, String hash) {
    // Implementación con biblioteca BCrypt de Spring Security
    // return new BCryptPasswordEncoder().matches(input, hash);
    return true; // Simplificado para el ejemplo
}
}

```

Herramientas para demostrar en clase

Herramienta	Uso en clase	Link
OWASP Threat Dragon	Modelado de amenazas visual y gratuito	https://owasp.org/www-project-threat-dragon/
OWASP ZAP	Escáner de vulnerabilidades web (demo en Semana 8)	https://www.zaproxy.org/
Have I Been Pwned	Verificar emails en filtraciones (apertura de clase)	https://haveibeenpwned.com
WebGoat (Docker)	Aplicación intencionalmente vulnerable para práctica	https://github.com/WebGoat/WebGoat
Juice Shop (Docker)	E-commerce vulnerable — ideal para proyecto	https://github.com/juice-shop/juice-shop
CVE Details	Base de datos de vulnerabilidades conocidas	https://www.cvedetails.com

REFERENCIAS EN FORMATO APA 7

- OWASP Foundation. (2021). *OWASP Top 10 — 2021: The ten most critical web application security risks*. OWASP. <https://owasp.org/www-project-top-ten/>
- Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.
- McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley Professional.
- Howard, M., & LeBlanc, D. (2002). *Writing secure code* (2nd ed.). Microsoft Press.
- Palazón Romero, J. M. (2013). *Programación segura de aplicaciones web*. Universitat Oberta de Catalunya. <https://openaccess.uoc.edu/server/api/core/bitstreams/e1dac4d3-d087-4be4-892d-c9342ff1762d/content>
- Ortega Candel, J. M. (2019). *Seguridad en aplicaciones web Java* (1a ed.). Ediciones de la U.
- National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63b>
- SEI CERT. (2023). *SEI CERT Oracle coding standard for Java*. Carnegie Mellon University. <https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>
- Hernández Bejarano, M., & Baquero Rey, L. E. (2020). *Ciclo de vida de desarrollo ágil de software seguro* (1a ed.). Editorial Los Libertadores.
- Gallagher, T., Landauer, L., & Jeffries, B. (2006). *Hunting security bugs*. Microsoft Press.
-

RECURSOS REALES Y LINKS ÚTILES

Documentación oficial

- **OWASP Top 10 2021:** <https://owasp.org/www-project-top-ten/>
- **OWASP Cheat Sheet Series:** <https://cheatsheetseries.owasp.org/>
- **NIST SP 800-63B (contraseñas):** <https://pages.nist.gov/800-63-3/sp800-63b.html>
- **CERT Secure Coding**
Java: <https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>
- **CWE Top 25 Most Dangerous Weaknesses:** https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html

Repositorios GitHub para esta sesión

- **WebGoat (entorno de práctica Java):** <https://github.com/WebGoat/WebGoat>
- **OWASP Juice Shop (e-commerce vulnerable):** <https://github.com/juice-shop/juice-shop>
- **OWASP Threat Dragon:** <https://github.com/OWASP/threat-dragon>

- **DVWA (PHP vulnerable):** <https://github.com/digininja/DVWA>
- **OWASP ESAPI Java (librería de seguridad):** <https://github.com/ESAPI/esapi-java-legacy>
- **Spring Security Samples:** <https://github.com/spring-projects/spring-security-samples>

Herramientas gratuitas para usar desde hoy

- **Have I Been Pwned:** <https://haveibeenpwned.com>
- **CVE Database NIST:** <https://nvd.nist.gov/>
- **Shodan (motor de búsqueda de dispositivos IoT):** <https://www.shodan.io>
- **OWASP ZAP Scanner:** <https://www.zaproxy.org/>

Cursos gratuitos complementarios

- **PortSwigger Web Security Academy** (100% gratuito, laboratorios interactivos): <https://portswigger.net/web-security>
- **OWASP WebGoat Labs:** <https://github.com/WebGoat/WebGoat>
- **Cybrary — Secure Coding:** <https://www.cybrary.it/>